

Développement Android avec Java

Chapitre 2 : Les ressources

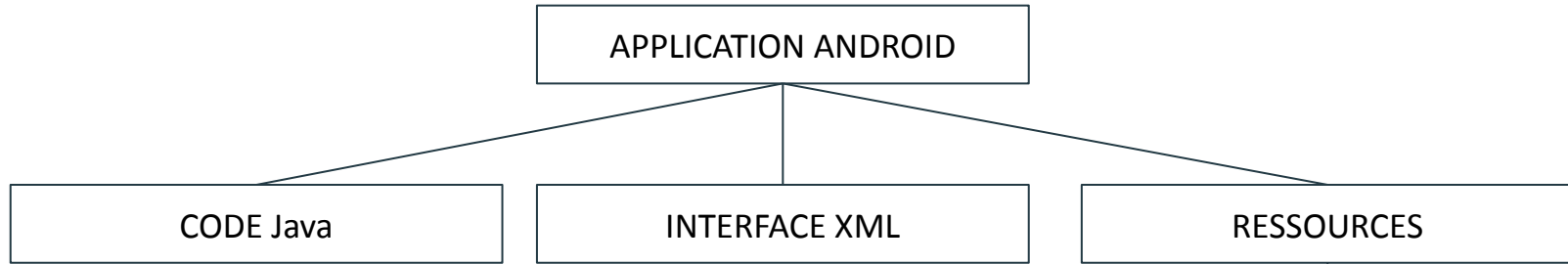


2026/2027

Objectif du chapitre



Comprendre comment Android organise et utilise les éléments nécessaires à une application : textes, interfaces, images, couleurs, dimensions et identifiants.



Une application peut avoir :

- le texte « Bienvenue »
- une image de logo
- une couleur principale
- une taille de texte
- une interface XML

Textes
Images
Couleurs
Dimensions
Styles

Une ressource est un élément utilisé par l'application mais géré séparément du code principal.



Pourquoi utiliser des ressources ?

Imaginez une application avec :

- 20 textes
- 10 couleurs
- plusieurs images
- plusieurs écrans
- différentes tailles d'écran
- plusieurs langues



Ressources

Textes
Images
Couleurs
Dimensions

l'application devient difficile à modifier et à maintenir.

Les ressources permettent de séparer les données utilisées par l'interface du code de l'application.



Organisation du dossier res/

 res/

drawable/	→ images
layout/	→ écrans (XML)
menu/	→ menus
mipmap/	→ icône de l'application
values/	→ textes, couleurs, dimensions, styles

Chaque ressource va dans le sous-dossier correspondant à son type.

Les types de ressources

Type	Fichier / dossier
Chaînes	values/strings.xml
Couleurs	values/colors.xml
Dimensions	values/dimens.xml
Images	drawable/
Écrans	layout/
Menus	menu/
Styles / thèmes	values/styles.xml
Préférences, etc.	xml/

Une chaîne
@string/bonjour

Une couleur
@color/primary

Une image
@drawable/logo

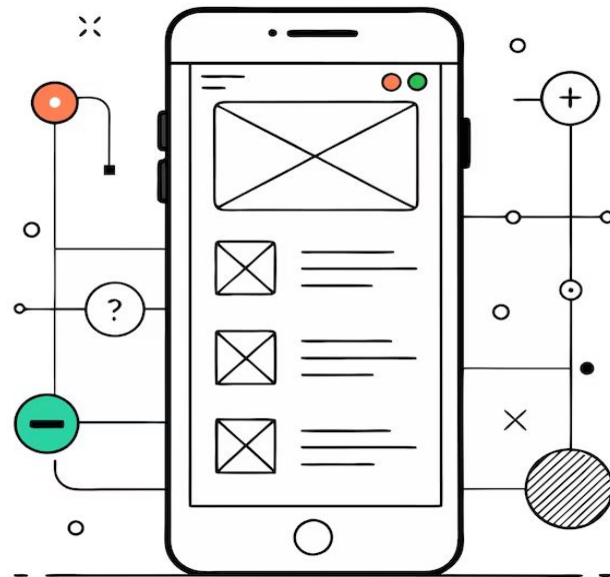
Un layout
@layout/activity_main

Une ressource Android est généralement
référéncée avec la forme @type/nom.

Les layouts

Un layout décrit l'organisation visuelle d'un écran.

- la position des composants
- leur taille
- leur contenu
- leur organisation



Un layout est une ressource

```
res/  
└─ layout/  
    └─ activity_main.xml
```

```
<TextView  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Bonjour" />
```



Attention au texte "hardcoded"

Layout.XML :

```
<TextView
```

```
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Bonjour" />
```

Le texte "Bonjour" est directement écrit dans le layout.



- Le même texte peut être utilisé à plusieurs endroits.
- modifier ou gérer plusieurs langues
-



```
res/  
└── values/  
    └── strings.xml
```

strings.XML :

```
<resources>  
    <string name="bonjour">  
        Bonjour  
    </string>  
</resources>
```

Layout.XML:

```
<TextView
```

```
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="@string/bonjour" />
```


@string/nom_de_la_chaine

```
<string name="titre">  
  Mon application  
</string>
```

```
android:text="@string/titre"
```

@string/titre

nom

type

référence

Android utilise le même principe pour différents types de ressources.

```
@string/bonjour  
@color/primary  
@drawable/logo  
@layout/activity_main
```

Exemples :

```
android:text="@string/bonjour"  
android:textColor="@color/primary"  
android:src="@drawable/logo"
```

@ signifie que nous faisons référence à une ressource Android.

Les ressources et Java

Android génère une classe appelée R : elle permet d'accéder aux ressources.

Exemples :

R.string.bonjour
R.color.primary
R.drawable.logo
R.layout.activity_main
R.id.monBouton

XML

Java

@string/bonjour → R.string.bonjour

@color/primary → R.color.primary

@drawable/logo → R.drawable.logo



Internationalisation

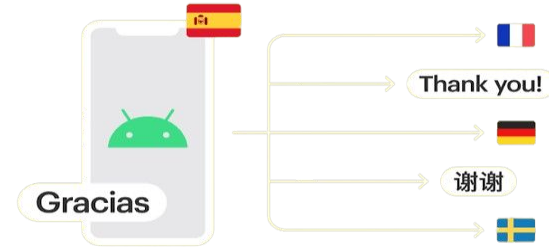
Une application peut parler plusieurs langues

Français : Bienvenue

Anglais : Welcome

values/strings.xml → texte par défaut (ex : anglais)

values-fr/strings.xml → texte en français



Android choisit automatiquement le bon dossier selon la langue configurée sur le terminal.

Les dimensions

Dans une interface Android, nous devons définir des dimensions :

- largeur
- hauteur
- marges
- espaces
- taille du texte

```
android:layout_width="200dp"  
android:layout_height="50dp"
```



Les dimensions permettent de contrôler la taille et l'espacement des éléments de l'interface.



Les images

Une application utilise souvent des images :

- logo
- icône
- photo
- illustration
- image de fond

```
res/  
└─ drawable/  
    └─ logo.png  
    └─ photo.png  
    └─ illustration.png
```

XML:

```
<ImageView  
    android:layout_width="100dp"  
    android:layout_height="100dp"  
    android:src="@drawable/logo" />
```

Java:

```
ImageView image =  
findViewById(R.id.imageLogo);  
  
image.setImageResource(R.drawable.logo);
```

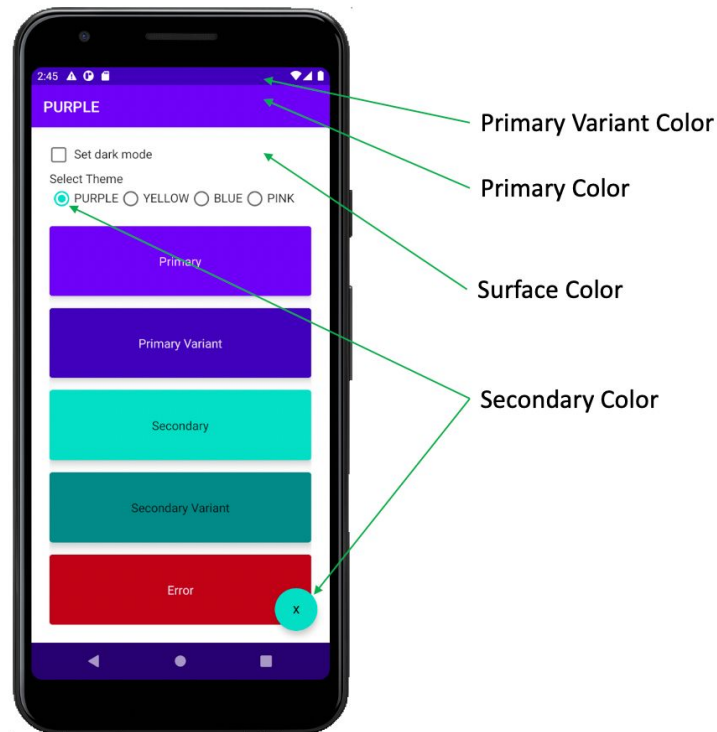


Les couleurs

res/
└─ values/
 └─ colors.xml

XML

```
<TextView  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="@string/bonjour"  
    android:textColor="@color/primary" />
```



Une interface contient des Views

Un écran Android est composé de différents composants graphiques.

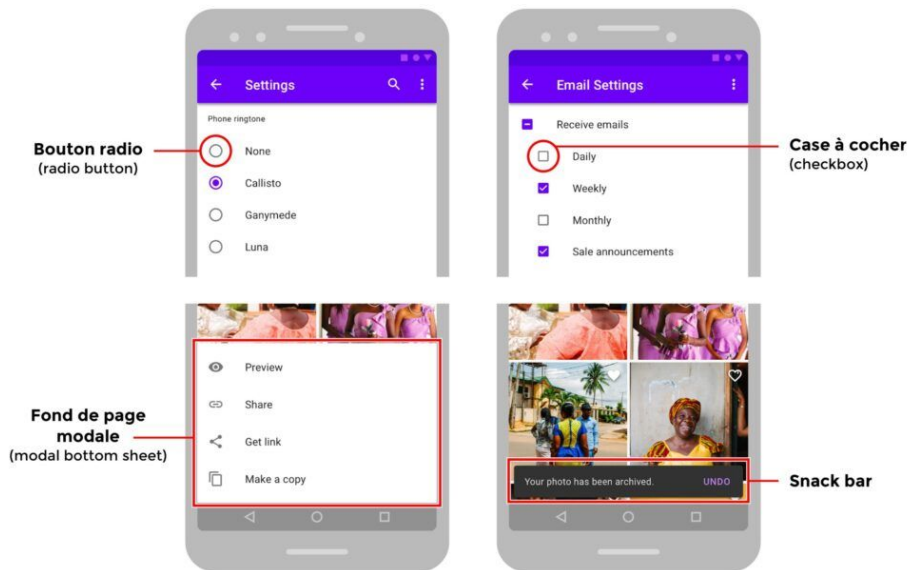
Écran

— TextView

— EditText

— Button

— ImageView



XML:

```
<TextView ... />
```

```
<EditText ... />
```

```
<Button ... />
```

```
<ImageView ... />
```

Référencer une View en Java

1- Déclarer un ID

```
<Button  
    android:id="@+id/btnValider"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="@string/valider" />
```

id permet d'identifier une View dans l'interface.
syntaxe **android:id="@+id/mon_identifiant**

2- XML → Java

XML
android:id="@+id/btnValider"



R.id.btnValider Android génère une référence

Java
Button bouton;

bouton = findViewById(R.id.btnValider);

Exemple

XML

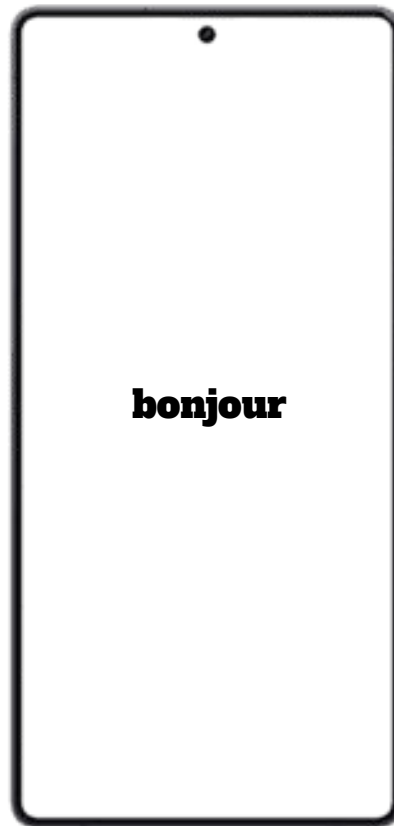
```
<TextView  
    android:id="@+id/txtMessage"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="@string/bonjour" />
```

Java

```
TextView text;
```

```
text = findViewById(R.id.txtMessage);
```

```
text.setText(R.string.bienvenue);
```



Mini-exercice Java

Comment récupérer ce bouton dans Java ?

```
<Button  
    android:id="@+id/btnConnexion"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="@string/connexion" />
```

Correction

```
Button boutonConnexion;  
  
boutonConnexion = findViewById(R.id.btnConnexion);
```

Ce qu'il faut retenir

res/ regroupe toutes les ressources, classées par type de sous-dossier

La classe R relie chaque ressource au code Java

Syntaxe : @type/nom en XML, R.type.nom en Java

Les qualificatifs (-fr, -land, -w820dp...) adaptent l'appli à la langue, l'écran, l'orientation

Toujours préférer une ressource à un texte/couleur/dimension "en dur"